

📌 Ultimate Python Roadmap for Machine Learning, Data Science & AI

Goal: Become highly proficient in Python for Machine Learning, Data Science, AI, Automation, and Data Visualization before learning NumPy.

Estimated Duration: 12–18 weeks (depending on study time)

Recommended Daily Schedule:

- ☐ Theory: 1–2 hours
 - ☐ Coding: 2–3 hours
 - ☐ Problem Solving: 1–2 hours
 - ☐ Revision: 30 minutes
-

Phase 0 — Programming Foundations

Goal

Build a solid understanding of how programming and Python work before writing code.

Topics

0.1 Introduction to Programming

- What is programming?
- Why programming exists
- Programming languages
- Types of programming languages
- Low-level vs High-level languages

0.2 How Computers Execute Programs

- CPU
- RAM
- Storage
- Instructions
- Binary
- Machine language

0.3 Compiler vs Interpreter

- Compilation
- Interpretation

- Bytecode
- Python Virtual Machine (PVM)

0.4 Python Introduction

- History
- Philosophy
- Zen of Python
- Features
- Advantages
- Limitations
- Applications

0.5 Python Versions

- Python 2 vs Python 3
- Current versions
- Release cycle

0.6 Installing Python

- Windows
- Linux
- macOS

0.7 IDEs

- VS Code
- PyCharm
- Jupyter Notebook (overview only)

0.8 Python Execution

- Interactive Mode (REPL)
- Script Mode
- Running from Terminal
- Running from VS Code

0.9 First Python Program

0.10 Comments

- Single-line
- Multi-line
- Docstrings

0.11 Input and Output

0.12 print() Function

- sep
 - end
 - escape sequences
-

Assignment

- Build a simple calculator
 - Build a greeting program
 - Build a unit converter
 - Build a simple age calculator
 - Build a temperature converter
-

Mini Project

Student Information Card Generator

Phase 1 — Python Basics

Goal

Understand Python syntax and fundamental building blocks.

Topics

Variables

Identifiers

Keywords

Literals

- Numeric
- Boolean
- String
- None

Data Types

- int
- float
- complex
- bool
- str
- bytes
- bytearray
- memoryview
- NoneType

Dynamic Typing

Type Checking

- type()
- isinstance()

Type Conversion

Multiple Assignment

Unpacking

Formatted Strings

Escape Characters

User Input

Python Memory Basics

Mini Project

Student Grade Calculator

Phase 2 — Operators & Expressions

Topics

Arithmetic Operators

Comparison Operators

Logical Operators

Assignment Operators

Identity Operators

Membership Operators

Bitwise Operators

Operator Precedence

Expression Evaluation

Short Circuit Evaluation

Truthiness & Falsiness

Mini Project

Scientific Calculator

Phase 3 — Control Flow

Topics

if

elif

else

Nested if

match-case

while

for

range()

break

continue

pass

Nested Loops

Loop else

Pattern Printing

Decision Trees

Mini Project

ATM Simulation

Phase 4 — Strings

Topics

String Creation

Unicode

ASCII

Indexing

Negative Indexing

Slicing

Immutability

Memory Model

Every String Method

String Searching

Formatting

Old Formatting

format()

f-Strings

Encoding

Decoding

Regular String Problems

Palindrome

Anagram

Frequency Counter

Compression

Mini Project

Password Strength Checker

Phase 5 — Collections

Lists

- Creation
- Indexing
- Slicing
- Nested Lists
- Methods
- Copying
- Sorting
- Searching

Tuples

- Packing
- Unpacking
- Methods

Sets

- Operations
- Methods
- Frozen Sets

Dictionaries

- Creation
- Methods
- Nested Dictionaries
- Dictionary Views

Complexity Analysis

Time Complexity of every operation

Mini Project

Inventory Management System

Phase 6 — Functions

Topics

Functions

Parameters

Arguments

Keyword Arguments

Positional Arguments

Default Parameters

Variable Length Arguments

Return

Scope

Namespaces

LEGB Rule

Recursion

Lambda

Higher Order Functions

map()

filter()

reduce()

zip()

enumerate()

Functions as Objects

Closures

Mini Project

Bank Management System

Phase 7 — Modules & Packages

Topics

Import

from import

Alias

Custom Modules

Packages

init.py

pip

venv

Requirements.txt

Useful Built-in Modules

- math
- random
- statistics
- datetime
- os
- pathlib

Mini Project

Utility Toolkit

Phase 8 — File Handling

Topics

Opening Files

Modes

Reading

Writing

Appending

Seeking

CSV

JSON

Binary Files

Context Managers

with Statement

Large File Processing

Mini Project

Student Database System

Phase 9 — Exception Handling

Topics

Errors

Exceptions

try

except

else

finally

raise

Assertions

Custom Exceptions

Logging Basics

Mini Project

Secure Login System

Phase 10 — Object-Oriented Programming

Topics

Classes

Objects

Attributes

Methods

Constructors

Instance Variables

Class Variables

Static Methods

Class Methods

Encapsulation

Inheritance

Polymorphism

Abstraction

Composition

Aggregation

Magic Methods

Property Decorators

Operator Overloading

Method Resolution Order (MRO)

Multiple Inheritance

Mini Project

Library Management System

Phase 11 — Advanced Python

Topics

Iterators

Generators

Yield

Generator Expressions

Decorators

Closures

Comprehensions

NamedTuple

Dataclasses

collections Module

deque

Counter

defaultdict

OrderedDict

ChainMap

heapq

itertools

functools

typing

Type Hints

Annotations

Regular Expressions

Performance Optimization Basics

Memory Optimization

Mini Project

Task Scheduler

Phase 12 — Python for Data Science Preparation

Goal

Master Python tools required before NumPy.

Topics

Working with Files

CSV Basics

JSON Basics

Dataset Handling

File Paths

pathlib

os

glob

math

random

statistics

datetime

time

collections

typing

Data Cleaning with Pure Python

Reading Large Files

Basic Data Aggregation

Sorting Data

Filtering Data

Searching Data

Memory Efficient Programming

Mini Project

Employee Data Analyzer

Phase 13 — Python Capstone Project

Build a Complete Data Processing Application

Features

- Read CSV files
 - Clean data
 - Validate data
 - Search records
 - Filter records
 - Sort records
 - Export reports
 - Error handling
 - Logging
 - Modular architecture
 - Object-oriented design
 - Documentation
 - Command-line interface
-

Every Topic Should Include

For every single topic in every phase:

- ☐ Goal
 - ☐ Explanation
 - **?** Why it exists
 - ☐ Real-world usage
 - ☐ Internal working (where appropriate)
 - ☐ Syntax
 - ☐ Examples (simple → advanced)
 - ☐ Common mistakes
 - ☐ Edge cases
 - ☐ Interview questions
 - ✓ Best practices
 - ☐ PEP 8 coding standards
 - ☐ Comparison with related concepts
 - ☐ Summary
 - ☐ Memory tricks
-

Practice After Every Topic

Level 1 — Warm-up (5 Questions)

Simple syntax practice.

Level 2 — Beginner (5 Questions)

Basic coding problems.

Level 3 — Intermediate (5 Questions)

Logic-building exercises.

Level 4 — Challenging (5 Questions)

More complex coding tasks.

Level 5 — Problem Solving (5 Questions)

Algorithmic and real-world problems.

Rule: Attempt the questions first. Solutions are discussed only after your submission.

End of Every Phase

Each phase concludes with:

- 5 integrated assignments
 - 1 mini project using only learned concepts
 - Revision notes
 - Cheat sheet
 - Common interview questions
 - Frequently asked coding questions
 - Tricky MCQs
 - Output prediction questions
 - Time complexity review (where applicable)
-

Problem-Solving Framework

For every coding problem, follow this process:

1. Understand the problem statement
 2. Identify inputs and expected outputs
 3. Note constraints and edge cases
 4. Design a brute-force solution
 5. Improve to a better approach
 6. Optimize for time and space complexity
 7. Perform a dry run with sample input
 8. Write clean, PEP 8-compliant code
 9. Test with normal and edge-case inputs
 10. Analyze complexity and summarize key takeaways
-

Coding Standards (PEP 8)

Throughout the roadmap, you'll consistently practice:

- Descriptive variable and function naming
 - Proper indentation (4 spaces)
 - Sensible file and project organization
 - Meaningful comments and docstrings
 - Readable, maintainable code
 - Avoiding duplication (DRY principle)
 - Writing reusable functions
 - Basic testing and debugging habits
-

Completion Outcome

After successfully finishing all 14 phases, you will be able to:

- Write clean, professional Python code from scratch.
- Solve medium-to-advanced Python programming problems confidently.
- Build command-line applications using procedural and object-oriented programming.
- Work with files, JSON, CSV, exceptions, and Python's standard library.
- Understand advanced Python concepts such as generators, decorators, comprehensions, iterators, type hints, and regular expressions.
- Develop strong problem-solving skills and follow industry-standard coding practices.

At this point, **you are fully prepared to start NumPy**, which is the foundation for numerical computing and the next major step toward Machine Learning, Data Science, and AI. From there, a natural progression is:

NumPy → Pandas → Matplotlib → Seaborn → Scikit-learn → Deep Learning (TensorFlow/PyTorch)